Databases and Applications

Unit III

Pooja T S

Computer Applications

Databases and Applications

Unit III

MongoDB CRUD: insert, update, delete

Pooja T S

Computer Applications

Databases and Applications

Introduction to MongoDB

PES UNIVERSITY

- **What is MongoDB?**
  - A NoSQL, document-oriented database storing data as `BSON` (Binary JSON).
  - Uses flexible, schema-less collections instead of rigid tables.
  - Designed for scalability, high availability, and handling semi-structured data.
- **Where MongoDB is used**
  - Web apps, analytics, IoT data, product catalogs, student records, logs.
  - Best suited where structure evolves frequently.

Databases and Applications

MongoDB vs SQL – Quick Comparison

PES UNIVERSITY

- **Key Concepts**
  - SQL: tables, rows, fixed schemas.
  - MongoDB: collections, documents, flexible schemas.
- **Why choose MongoDB?**
  - Easy to store nested fields (arrays, objects).
  - No schema migrations required.
  - Faster prototyping for modern applications.

Databases and Applications

Mongo Shell – Getting Started

PES UNIVERSITY

- **What is the Mongo Shell?**
  - An interactive command-line interface to communicate with MongoDB.
  - Supports JavaScript-like command syntax.
  - Used for administration, querying, and CRUD operations.
- **Basic Shell Commands**
  - `mongo` — start shell.
  - `show dbs` — list databases.
  - `use college` — switch/create database.
  - `show collections` — list collections.

Databases and Applications

MongoDB Data Types – Overview

PES UNIVERSITY

- **MongoDB supports flexible data types:**
  - **String** – Most common text data type. (`"ABC"`)
  - **Number** – `int`, `long`, `double`. (`21`, `45.6`)
  - **Boolean** – `true` / `false`.
- Used in documents to represent text, numeric values, flags, and simple state information.

Databases and Applications

MongoDB Data Types – Complex Types

- **Advanced supported types:**
  - **Array** – Stores multiple values. (`["Python", "SQL"]`)
  - **Object / Embedded Document** – Nested fields. (`{ city: "Bengaluru"}`)
  - **ObjectId** – Auto-generated unique identifier.
- Supports flexible, schema-less structure with nested and repeated fields.

MongoDB Data Types – Additional Types

- **Other important types:**
  - **Date** – Stores timestamps. (`ISODate(...)`)
  - **Null** – Missing / unknown value.
  - **Binary Data** – Images, PDFs, encoded content.
- These types help store structured, semi-structured, and application-specific data.

Databases and Applications

MongoDB Documents – Introduction

PES UNIVERSITY

- **MongoDB stores data as BSON (Binary JSON).**
  - Accepts standard JSON input in the shell.
  - Internally converts JSON to efficient BSON.
  - Supports nested fields and arrays naturally.
- **A document is a collection of key–value pairs.**
  - Keys are strings; values may be any valid datatype.
  - Fields may contain nested documents.
  - Order of fields is preserved but not required.

Databases and Applications

How to Structure a MongoDB Document

- **Document = key–value structure**
  - Uses `JSON`-like syntax stored as `BSON`.
  - Fields may be strings, numbers, arrays, or embedded docs.
  - No fixed schema — fields can differ across documents.
- **Example student document**

```
{ name: "Aarathi", age: 21, dept: "CSE" }
```

Databases and Applications

Sample Document – Good Structure

- **Document with nested fields:**

```
{
  name:  "Riya",
  age:  20,
  skills:  ["Python","MongoDB"],
  address:  { city:  "Bengaluru", pincode:  560050
}
}
```

- **Why this structure works well:**
  - Natural representation of real-world objects.
  - Easy to extend with new fields.
  - Supports deep queries using dot notation.

Databases and Applications

Rules for Writing MongoDB Queries

- **Important rules:**
  - Keys must always be in double quotes when nested.
  - Queries are always written inside { }.
  - Multiple conditions use `$and`, `$or`, or implicit AND.
- **Examples**

```
db.students.find({ age:  21, dept:  "CSE" })
db.students.find({ $or:  [ {dept:"CSE"}, {age:22}
] })
```

Databases and Applications

CRUD — Insert Operations

- **Insert a single document**

```
db.students.insertOne({ name: "Amit", age: 21,
dept: "CSE" })
```

- **Insert multiple documents**

```
db.students.insertMany([ { name: "Riya", age:
20, skills: ["Python"] }, { name: "Kiran", age:
22, skills: ["SQL","Java"] } ])
```

- **Where inserts are used**
  - Adding new student profiles.
  - Importing bulk data from apps/forms.

Databases and Applications

CRUD — Read (Find) Operations

- **Read all documents**

```
db.students.find()
```

- **Find with filters**

```
db.students.find({ dept:  "CSE" })
db.students.find({ age:  { $gt:  20 } })
```

- **Projection (show selected fields)**

```
db.students.find({}, { name:  1, dept:  1 })
```

Databases and Applications

CRUD — Update Operations

- **Update one document**

```
db.students.updateOne( { name:  "Amit" }, { $set:
{ age:  22 } } )
```

- **Update many documents**

```
db.students.updateMany( { dept:  "CSE" }, { $inc:
{ age:  1 } } )
```

- **Why updates are essential**
  - Correcting records, app-based profile updates.

Databases and Applications

CRUD — Delete Operations

- **Delete a single document**

```
db.students.deleteOne({ name:  "Kiran" })
```

- **Delete multiple documents**

```
db.students.deleteMany({ dept:  "CSE" })
```

- **Important Note**
  - Deletions are permanent — use filters carefully.
  - Ideal for removing inactive user profiles.

Thank You

Pooja T S

Assistant Professor

Department of Computer Applications

poojats@pes.edu

080-26721983 Extn: 391